

Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Інститут комп'ютерних систем
Кафедра інформаційних систем

Лабораторна робота №4

З дисципліни: «Програмування мобільних пристроїв на базі ОС Android»

Тема: «Локальна база даних Room та Repository в Android-застосунку»

Виконала: студентка групи AI-234

Левчук Є.В.

Перевірили: Годовиченко М.А.

Соловйова Д.В.

Одеса 2026

Мета роботи: набуття практичних навичок із організації локального збереження даних в Android-застосунку за допомогою бібліотеки Room. У ході роботи студент опановує архітектурний підхід із використанням шару Repository, вчиться відокремлювати логіку роботи з базою даних від бізнес-логіки застосунку, а також переписує ViewModel таким чином, щоб вона отримувала реактивні дані з локальної бази замість колекцій у пам'яті.

Завдання: доопрацювання застосунку «Моя бібліотека», створеного в лабораторній роботі 3. До проєкту необхідно додати бібліотеку Room і налаштувати процесор символів KSP для генерації коду під час компіляції. Клас Book потрібно перетворити на Entity-сутність бази даних, створити DAO-інтерфейс з операціями читання, додавання, оновлення та видалення, а також клас RoomDatabase для доступу до бази. Окремим завданням є реалізація Repository як проміжного шару між базою даних і ViewModel та переписування самої ViewModel так, щоб усі операції зі зміни даних виконувались асинхронно через корутини. Після виконання роботи застосунок повинен зберігати дані між сеансами – після закриття та повторного відкриття книги мають залишатися в базі.

ХІД РОБОТИ

Клас Book із лабораторної роботи 3 доопрацьовано шляхом додавання анотацій Room. Анотація @Entity з параметром tableName вказує що цей клас відповідає таблиці «books» у базі даних SQLite. Анотація @PrimaryKey з параметром autoGenerate = true означає що Room автоматично генерує унікальний числовий ідентифікатор для кожного нового запису. Значення за замовчуванням id = 0 є необхідним для коректного автогенерування – Room сприймає нульовий id як сигнал що запис новий і потребує присвоєння ідентифікатора.

Лістинг 1 – Entity-клас Book

```
@Entity(tableName = "books")
data class Book(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val title: String,
    val author: String,
    val year: Int,
    val rating: Float = 0f
)
```

Створено інтерфейс BookDao з анотацією @Dao, для якого Room автоматично генерує повну реалізацію під час компіляції. Метод getAllBooks() повертає Flow<List<Book>> – реактивний потік даних, завдяки якому щоразу коли таблиця змінюється Room автоматично випускає оновлений список і екран Compose перемальовується без жодного додаткового коду. Методи insertBook, updateBook та deleteBook є suspend-функціями і виконуються асинхронно у корутині. Для оновлення лише поля рейтингу реалізовано окремий метод updateRating із прямим SQL-запитом через анотацію @Query.

Лістинг 2 – DAO-інтерфейс BookDao

```
@Dao
interface BookDao {
    @Query("SELECT * FROM books ORDER BY id ASC")
    fun getAllBooks(): Flow<List<Book>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertBook(book: Book)

    @Update
```

```

suspend fun updateBook(book: Book)

@Delete
suspend fun deleteBook(book: Book)

@Query("UPDATE books SET rating = :rating WHERE id = :id")
suspend fun updateRating(id: Int, rating: Float)
}

```

Створено абстрактний клас `BookDatabase` що успадковує `RoomDatabase`. Анотація `@Database` перераховує всі `Entity`-класи та версію схеми бази. Реалізовано патерн `Singleton` через `companion object` з анотацією `@Volatile` – це гарантує що в будь-який момент існує лише один екземпляр бази даних у всьому застосунку, що є критичним для коректної роботи з `SQLite`.

Лістинг 3 – Клас `BookDatabase`

```

@Database(entities = [Book::class], version = 1, exportSchema = false)
abstract class BookDatabase : RoomDatabase() {
    abstract fun bookDao(): BookDao

    companion object {
        @Volatile private var INSTANCE: BookDatabase? = null

        fun getDatabase(context: Context): BookDatabase {
            return INSTANCE ?: synchronized(this) {
                Room.databaseBuilder(
                    context.applicationContext,
                    BookDatabase::class.java,
                    "book_database"
                ).build().also { INSTANCE = it }
            }
        }
    }
}

```

Реалізовано клас `BookRepository` як проміжний шар між `DAO` і `ViewModel`. Його роль полягає у тому щоб приховати від `ViewModel` деталі того звідки беруться дані. Властивість `allBooks` делегує `Flow` безпосередньо з `DAO`, а методи `addBook`, `updateBook` і `deleteBook` конструюють або знаходять об'єкт `Book` і передають його до відповідних функцій `DAO`. Усі методи що змінюють дані є `suspend`-функціями і викликаються виключно з корутин.

Лістинг 4 – Фрагмент `BookRepository`

```

class BookRepository(private val bookDao: BookDao) {
    val allBooks: Flow<List<Book>> = bookDao.getAllBooks()

    suspend fun addBook(title: String, author: String, year: Int) {
        bookDao.insertBook(Book(title = title, author = author, year = year))
    }

    suspend fun deleteBook(id: Int) {
        val existing = bookDao.getBookById(id) ?: return
        bookDao.deleteBook(existing)
    }
}

```

ViewModel переписано так щоб вона приймала Repository через конструктор. Колекція книг тепер представлена як StateFlow отриманий з Flow бази даних через оператор stateIn. Параметр SharingStarted.WhileSubscribed(5000) означає що Flow залишається активним ще п'ять секунд після того як останній підписник відписався – це запобігає зайвому перезапуску при повороті екрана. Усі методи зміни даних запускають корутину через viewModelScope.launch і делегують роботу до Repository.

Лістинг 5 – Оновлена BookViewModel

```

class BookViewModel(private val repository: BookRepository) : ViewModel() {

    val books: StateFlow<List<Book>> = repository.allBooks.stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5000),
        initialValue = emptyList()
    )

    fun addBook(title: String, author: String, year: Int) {
        viewModelScope.launch { repository.addBook(title, author, year) }
    }

    fun deleteBook(id: Int) {
        viewModelScope.launch { repository.deleteBook(id) }
    }
}

```

В MainActivity створено екземпляр бази даних через BookDatabase.getDatabase(), з якого отримано DAO і передано до Repository. Щоб ViewModel отримала Repository через конструктор створено BookViewModelFactory – власну реалізацію ViewModelProvider.Factory. Завдяки

цьому lifecycle-система Android коректно управляє життєвим циклом ViewModel. Екрани застосунку та навігація залишились без змін – це є підтвердженням правильності архітектурного підходу з Repository.

Після запуску застосунку перевірено коректність збереження даних за допомогою вбудованого інструменту Android Studio – Database Inspector. У вкладці App Inspection відкрито Database Inspector, де в базі book_database у таблиці books відображено доданий запис із полями id, title, author, year та rating. Після повторного запуску застосунку дані залишились у базі, що підтверджує успішну реалізацію постійного збереження.

Демонстрація роботи застосунку

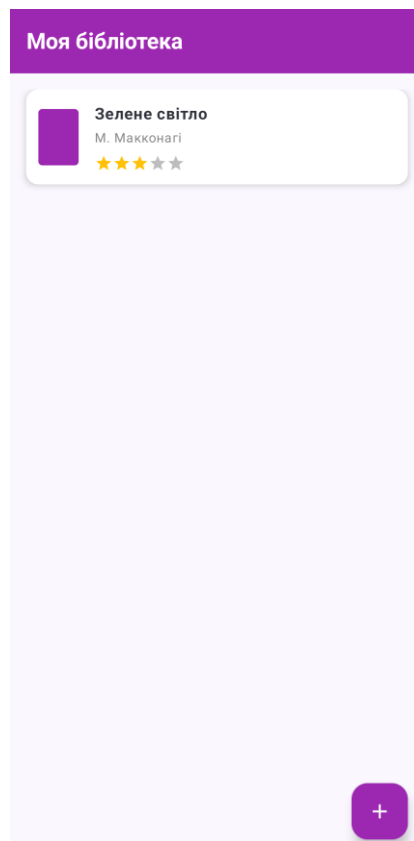


Рисунок 1 – Головний екран із книгами збереженими в базі даних

9:58

Нова книга

Назва книги

Автор

Рік видання

Додати

Скасувати

Рисунок 2 – Форма додавання книги

Pixel 6 > com.example.lab3

Database Inspector Network Inspector Background Task Inspector

Databases

book_database SQLiteDatabase

books

room_master_table

books

Live updates

id	title	author	year	rating
1	Зелене світло	M. Махоніні	2021	3.0

Рисунок 3 – Вміст таблиці books у Database Inspector

ВИСНОВОК

У ході виконання лабораторної роботи застосунок «Моя бібліотека» успішно доопрацьовано шляхом інтеграції локальної бази даних Room. Реалізовано повний стек роботи з даними: Entity-клас як опис таблиці, DAO-інтерфейс із SQL-операціями що генеруються автоматично, RoomDatabase як точка доступу до бази та Repository як проміжний шар абстракції між базою і ViewModel.

Переписана ViewModel отримує реактивний потік даних через Flow що забезпечує автоматичне оновлення інтерфейсу при будь-яких змінах у базі. Усі операції зміни даних виконуються асинхронно через Kotlin Coroutines у viewModelScope що унеможливорює блокування головного потоку. Практично підтверджено що дані зберігаються між сеансами роботи застосунку – після закриття та повторного відкриття книги залишаються в базі, що й є головним результатом даної лабораторної роботи.

ДОДАТОК

Посилання на репозиторій: <https://github.com/kanava2000/Lab4.git>